

Ingeniería y Calidad de Software

Resumen de Estudio - Primer Parcial | UTN FRC - Sistemas de Información

Unidad 1: Ingeniería de Software en Contexto

1. ¿QUÉ ES LA INGENIERÍA DE SOFTWARE?

El software no es solo código; abarca desde representaciones abstractas (requerimientos) hasta concretas (código ejecutable, documentación, configuraciones). Según **Sommerville**, es una disciplina que se ocupa de *todos los aspectos de la producción de software*, desde su concepción inicial hasta la operación y el mantenimiento.

La Crisis del Software

Dificultades históricas en la planificación, estimación de costos y calidad frente a una **demanda explosiva de sistemas complejos**. Se subestimaba la verdadera complejidad (software es abstracto e intangible frente al crecimiento físico del hardware).

No Silver Bullet (Brooks)

Establece que no existe una única tecnología o práctica que mejore la productividad de golpe. Divide las dificultades en: **Esenciales** (inherentes a la naturaleza del software: complejidad, invisibilidad, mutabilidad) y **Accidentales** (problemas técnicos superables).

2. DISCIPLINAS DE LA INGENIERÍA DE SOFTWARE

No todo es programar. La ingeniería de software moderna requiere un enfoque integral compuesto por tres tipos de disciplinas (vinculación Proceso-Proyecto-Producto):

Tipo	Foco	Incluye
Técnicas	"Hacer el software"	Requerimientos, Análisis, Diseño, Implementación, Pruebas, Despliegue.
Gestión	"Organizar el proyecto"	Planificación de Proyecto, Monitoreo y Control de Proyectos.
Soporte	"Controlar calidad e integridad"	Gestión de Configuración (SCM), Aseguramiento de Calidad, Auditorías, Métricas.

3. MODELOS DE PROCESO Y CICLOS DE VIDA

La elección del ciclo de vida depende de las características del producto y del grado de incertidumbre.

Procesos Definidos vs. Empíricos:

Los procesos *definidos* (tradicionales/en cascada) asumen que la ingeniería es predecible mediante especificaciones iniciales estáticas. Los procesos *empíricos* (ágiles) aceptan la alta variabilidad e incertidumbre, gestionándolas mediante tres pilares: **Transparencia, Inspección y Adaptación**.

- **Predictivos / Cascada:** Adecuados cuando los requisitos son claros, fijos y hay baja incertidumbre tecnológica. Desventaja: producto visible recién al final, alto riesgo de que no sirva al cliente.
- **Iterativos e Incrementales:** Construyen el sistema por partes (incrementos) y refinan (iteraciones). Facilitan la validación temprana.

Unidad 2: Gestión Lean Ágil de Productos de Software

1. REQUERIMIENTOS ÁGILES: USER STORIES (HISTORIAS DE USUARIO)

Forma ágil de expresar requerimientos centrados en el valor que aportan al usuario, más que en la especificación técnica detallada. Siguen el principio de las **3 Cs de Ron Jeffries**:

- **Card (Tarjeta):** Representación física o digital de la necesidad. Captura la intención ("*Como [rol] quiero [acción] para [beneficio]*").
- **Conversation (Conversación):** El diálogo continuo entre desarrolladores, QA y el Product Owner para afinar el detalle y resolver dudas técnicas.
- **Confirmation (Confirmación):** Los *Criterios de Aceptación* verificables que determinan cuándo la historia está terminada. Suelen usar el formato BDD (**Given / When / Then**).

Vertical Slicing

Estrategia de desglose en ambientes ágiles que busca entregar una funcionalidad completa de punta a punta (UI, Lógica, Base de Datos), habilitando la validación real del valor por el cliente, en lugar de entregar capas horizontales aisladas.

Criterio INVEST

Una buena User Story debe ser: **I**ndependiente, **N**egociable, **V**aliosa, **E**stimable, **S**mall (Pequeña) y **T**esteable.

2. DEFINICIÓN Y ENTREGAS DE PRODUCTO

MVP	Mínimo Producto Viable: Orientado al aprendizaje. Versión con las características justas para satisfacer a los primeros clientes y <i>validar hipótesis de negocio</i> con el menor esfuerzo posible.
MMF	Minimum Marketable Feature: Empaqueta el alcance mínimo necesario para ser lanzado al mercado generando un retorno o valor comercial directo.
MLP	Minimum Lovable Product: Integra elementos de usabilidad, diseño y UX desde el principio para generar lealtad y una conexión emocional en el usuario temprano.

3. ESTIMACIONES ÁGILES

Las metodologías ágiles estiman de forma **relativa** (comparando ítems entre sí) y no en horas exactas, reconociendo la incertidumbre.

- **Story Points (Puntos de Historia):** Unidad de medida relativa. Una estimación de Story Points amalgama tres factores: **Complejidad** del requerimiento, **Incertidumbre/Riesgo** técnico y **Volumen/Esfuerzo** de trabajo.
- **Secuencia de Fibonacci (1, 2, 3, 5, 8, 13, 21...):** Utilizada (ej. Planning Poker) porque refleja la Ley de Weber-Fechner (incertidumbre proporcional). A mayor tamaño del requerimiento, menor precisión para estimar (no tiene sentido debatir entre 20 o 21 puntos).

⚠ El Anti-patrón de la conversión a horas:

Traducir directamente Story Points a horas destruye la naturaleza empírica de la estimación ágil, genera fricción administrativa y distorsiona el cálculo real de la velocidad (Velocity) del equipo.

Unidad 3: Gestión de Configuración de Software (SCM)

1. CONCEPTO Y ROL DE SCM

La Gestión de Configuración de Software (SCM) es una **disciplina de soporte**. Su objetivo es establecer y mantener la integridad del producto a lo largo de todo su ciclo de vida. Es la base organizativa; sin un buen control de versiones y repositorios, las demás disciplinas (desarrollo, testing, despliegue) no pueden funcionar correctamente.

Ítem de Configuración (CI - Configuration Item):

Es el elemento base de SCM. Se define como *"cualquier artefacto del que se necesita disponer de trazabilidad o saber de su existencia durante el desarrollo del producto"* para resguardar su integridad. Ejemplos: código fuente, documentos de requerimientos, casos de prueba, scripts de bases de datos, manuales de usuario.

2. LAS 4 ACTIVIDADES BÁSICAS DE SCM

Actividad	Descripción
1. Identificación	Definir cuáles serán los Ítems de Configuración (CIs), cómo se van a nombrar (nomenclatura), sus versiones y cómo se estructurará el repositorio que los contiene.
2. Control de Cambios	Mecanismo formal para solicitar, evaluar, aprobar o rechazar modificaciones a los CIs que ya forman parte de una Baseline. Asegura que ningún cambio se aplique arbitrariamente.
3. Reporte de Estado	Contabilidad del estado de la configuración (Status Accounting). Provee trazabilidad: responde al <i>qué, quién, cuándo y por qué</i> cambió un artefacto.
4. Auditorías	Controles formales para verificar la calidad e integridad. FCA (Functional Configuration Audit) : Verifica que el software cumpla funcionalmente con los requerimientos. PCA (Physical Configuration Audit) : Confirma que todos los entregables físicos (código, manuales) están completos y estructurados.

3. CONCEPTOS CLAVE DE SCM

Baseline (Línea Base)

Estado formal, revisado y auditado de un conjunto de artefactos del proyecto en un momento dado. Impide modificaciones arbitrarias: una vez establecida, cualquier cambio a los CIs debe pasar obligatoriamente por un proceso formal de control de cambios. Garantiza la repetibilidad.

CCB (Comité de Control de Cambios)

Grupo de personas (Change Control Board) con la autoridad para evaluar, aprobar, rechazar y priorizar las solicitudes de cambio. Analizan el impacto técnico y económico antes de alterar una Baseline establecida.